

ПРАКТИЧЕСКАЯ РАБОТА 2

ЦВЕТОВЫЕ МОДЕЛИ

СОЗДАНИЕ И СОХРАНЕНИЕ ИЗОБРАЖЕНИЯ

Цель работы: ознакомление со способами создания и сохранения изображения, а так же с цветовыми моделями в Python с использованием внешней библиотеки PIL.

Подготовка к выполнению заданий

1. На компьютере должен быть выход в интернет.
2. Поместите в текущую папку набор цветных изображений.
3. Поместите не в текущую папку набор других цветных изображений.
4. Установите внешнюю библиотеку PIL.

Задание 1. Получение и изменение цвета пиксела

1. Получите и измените цвет пиксела методом `load()`.

Сначала получите информацию о цветовой модели открытого изображения:

```
>>> img = Image.open("foto.jpg")
>>> img.mode
'RGB'
```

Теперь получите цвет пиксела:

```
obj = img.load()
obj[25, 45] # Получаем цвет пикселя
(122, 86, 62)
# Задаем цвет пикселя (красный)
obj[25, 45] = (255, 0, 0)
img.show() # Просматриваем изображение (цвет пикселя будет
изменен на obj[25, 45], заданный выше.
```

2. Получите и измените цвет пиксела методами **get()** и **putpixel()**.

```
# Получаем цвет пикселя
img.getpixel((25, 45))
(255, 0, 0)
# Изменяем цвет пикселя
img.putpixel((25, 45), (0, 0, 255))
# Получаем цвет пикселя
img.getpixel((25, 45))
(0, 0, 255)
img.show() # Просматриваем изображение
```

Задание 2. Переход от одной цветовой модели изображения к другой

1. Преобразуйте изображение из режима **RGB** в **RGBA**, используя **split()** и **merge**.

split() – возвращает каналы изображения в виде кортежа.

merge(<Режим>, <Каналы>) – собирает изображение из каналов

(операция, обратная **split**).

```
>>> img = Image.open("foto.jpg")
>>> img.mode
'RGB'
>>> R, G, B = img.split()
>>> mask = Image.new("L", img.size, 128)
>>> img2 = Image.merge("RGBA", (R, G, B, mask) )
>>> img2.mode
'RGBA'
>>> img2.show()
```

2. Преобразуйте изображение из режима **RGB** в **RGBA**, используя **convert()**.

```
img = Image.open("foto.jpg")
```

```
img.mode
```

```
'RGB'
```

```
img2 = img.convert("RGBA")
```

```
img2.mode
```

```
'RGBA'
```

```
img2.show()
```

#прочитать изображение и конвертировать

его в градациях серого:

```
pil_im = Image.open('empire.jpg').convert('L')
```

3. Преобразуйте изображение **RGB** в формат **P**, указав смешивание цветов и адаптивную палитру в 128 цветов.

```
>>> img = Image.open("foto.jpg")
>>> img.mode
'RGB'
>>> img2 = img.convert("P", None,
Image.FLOYDSTEINBERG, Image.ADAPTIVE, 128)
>>> img2.mode
'p'
```

Задание 3. Сохранение изображения

Сохраните изображение в текущую папку.

Сохранение изображения в текущую папку в формате **JPEG**:

```
img.save("tmp.jpg")  
# в формате BMP:  
img.save("tmp.bmp", "BMP")  
f = open("tmp2.bmp", "wb")  
# Передаем файловый объект  
img.save(f, "BMP")  
f.close()
```

Задание 4. Создание нового изображения

1. Создайте черный квадрат.

```
img = Image.new("RGB", (100, 100))  
img.show() # Черный квадрат
```

2. Создайте красный квадрат.

```
img = Image.new("RGB", (100, 100), (255, 0, 0))  
img.show() # Красный квадрат
```

3. Создайте Зеленый квадрат.

```
img = Image.new("RGB", (100, 100), "green")  
img.show() # Зеленый квадрат
```

4. Создайте красный квадрат.

```
img = Image.new("RGB", (100, 100), "#f00")  
img.show() # тоже красный квадрат
```

5. Создайте белый квадрат.

```
img = Image.new("RGB", (100, 100), "white")  
img.show() # Белый квадрат
```

6. Создайте серый квадрат.

```
img = Image.new("RGB", (320, 240), "silver")  
img.show() # Серый квадрат
```

7. Создайте цветной прямоугольник.

```
img = Image.new("RGB", (320, 240), "rgb(205,
```

```
100,200) ")
```

```
img.show() # цветной прямоугольник
```

8. Создайте цветной прямоугольник (Каналы в процентах).

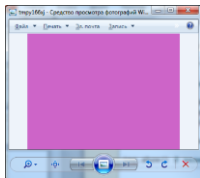
```
img = Image.new("RGB", (320, 240), "rgb(10%,  
100%,40%) ")
```

```
img.show() # цветной прямоугольник. Каналы в процентах
```

9. Создайте цветной прямоугольник заданного цвета. Затем поменяйте этот цвет.

```
img = Image.new("RGB", (640, 480), "rgb(205,  
100,200) ")
```

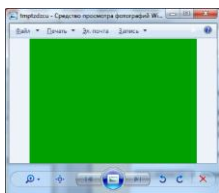
```
img.show() # сиреневый прямоугольник
```



```
for x in xrange(640):  
    for y in xrange(480):  
        img.putpixel((x,y), (0,160,0))
```

```
img.save("okno.png", "PNG")
```

```
img.show() # зеленый прямоугольник
```



10. Создайте прямоугольник заданного цвета (функциональная раскраска).

```
img = Image.new("RGB", (640, 480), "rgb(205,  
100,200) ")
```

```
img.show() # сиреневый прямоугольник
```

```
for x in xrange(640):  
    for y in xrange(480):
```

```

img.putpixel((x,y),
             x/3,(x+y)/6,y/3)

img.save("okno.png", "PNG")

img.show() # прямоугольник с функциональной раскраской

```



ЛИТЕРАТУРА К РАБОТЕ

1. Python Imaging Library (Fork) . – URL: <https://pypi.org/project/Pillow/> (дата обращения: 10.05.2023).
2. Прохоренок, Н. А. Python 3. Самое необходимое: Пособие / Прохоренок Н.А., Дронов В.А. - СПб:БХВ-Петербург, 2016. - 464 с. ISBN 978-5-9775-3631-8.-Текст:электронный).–URL: https://codernet.ru/books/python/python_3_samoe_neobxodimoe_proxorenok/ (дата обращения: 10.05.2023).
3. Python 3 и PyQt 6. Разработка приложений/ Н. А. Прохоренок, В. А. Дронов. -СПб.: Б:ХВ-Петербург, 2023.- 832 с.: ил.-(Профессиональное программирование). – URL: <https://coollib.net/b/617121-vladimir-dronov-python-3-i-pyqt-6-razrabotka-prilozheniy/readp?p=186&cnt=9000> (дата обращения: 10.05.2023).